Analyse the performance of different types of SVMs

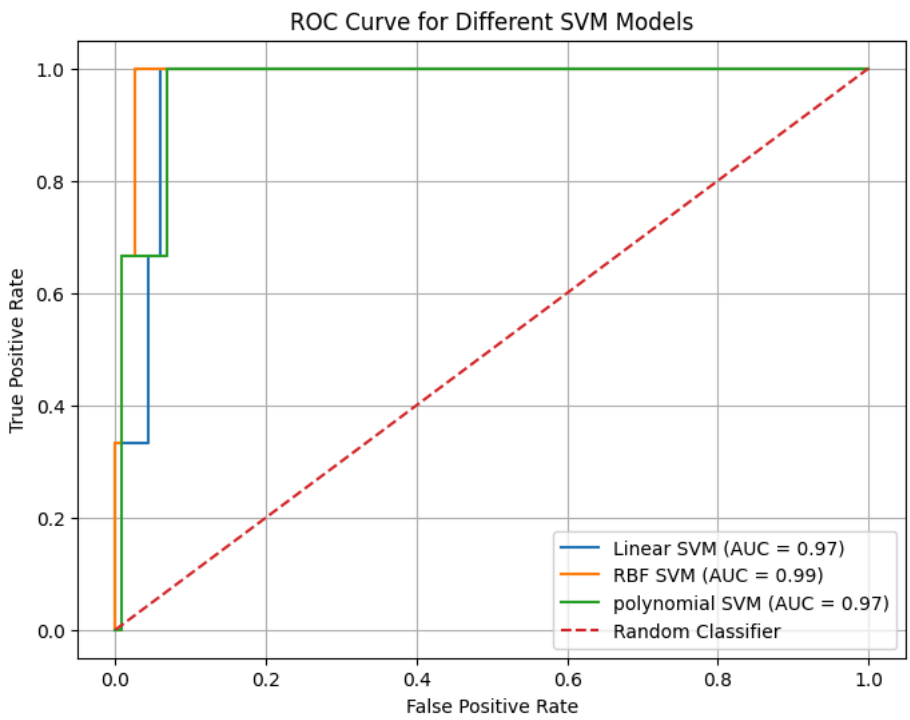
Name : -Swaraj Bharat Pawar div :- d1 roll no :- d047 sap id :-60009240227

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
```

```
X , y = fetch_olivetti_faces(return_X_y=True)
y_binary = (y==0).astype(int)

X_train,X_test,y_train , y_test = train_test_split(X,y_binary,test_size=0.3,random_state=42,stratify=y_binary)
```

```
# Define svm model
svm_models = {
    "Linear SVM": SVC(kernel="linear",probability=True),
    "RBF SVM": SVC(kernel="rbf",probability=True),
    "polynomial SVM": SVC(kernel="poly",degree=9,probability=True)
}
```

```
plt.figure(figsize=(8,6))

for name, model in svm_models.items():
    clf = make_pipeline(StandardScaler(),model)
    clf.fit(X_train,y_train)

    y_scores = clf.predict_proba(X_test)[:,1]

    fpr,tpr, _ = roc_curve(y_test,y_scores)
    roc_auc = auc(fpr,tpr)

    plt.plot(fpr,tpr,label=f"{name} (AUC = {roc_auc:.2f})")


plt.plot([0,1],[0,1],linestyle="--",label="Random Classifier")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve for Different SVM Models")
plt.legend()
plt.grid(True)
plt.show()
```



Orange is best as it has highest area under curve

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

Impact of Loss and C value on SVM implementation

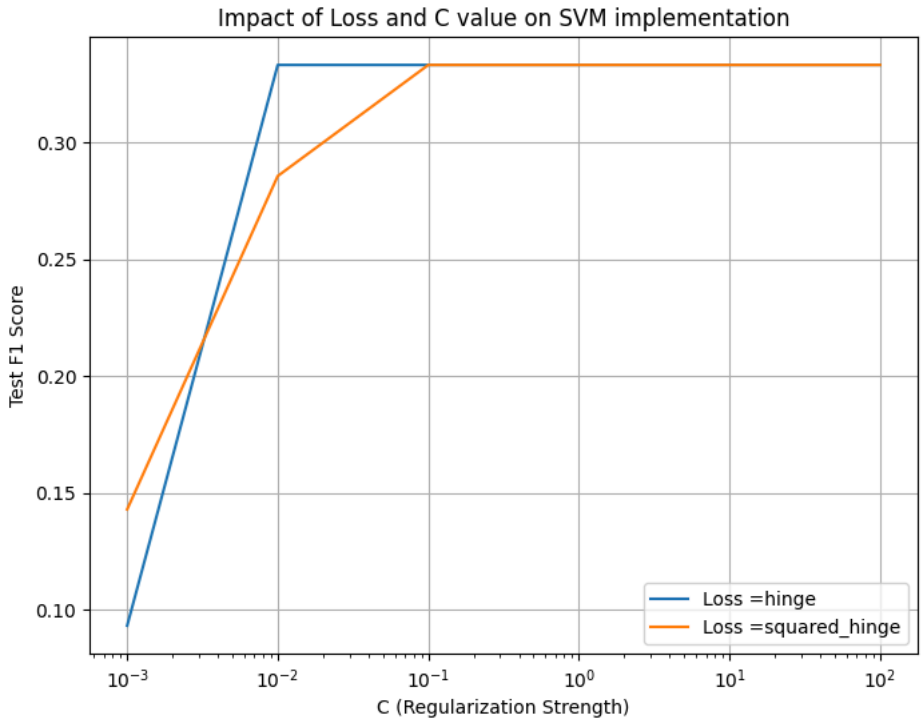
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.svm import LinearSVC
from sklearn.metrics import f1_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```
X , y = fetch_olivetti_faces(return_X_y=True)
y = (y==0).astype(int)

X_train,X_test,y_train , y_test = train_test_split(X,y_binary,test_size=0.3,random_state=42,stratify=y)
```

```
C_values = [0.001,0.01,0.1,1,10,100]
losses = ["hinge","squared_hinge"]
results = {loss:[] for loss in losses}
```

```
for loss in losses:
    for C in C_values:
      model = Pipeline([
          ("scaler",StandardScaler()),
          ("svm",LinearSVC(
              C=C,
              loss=loss,
              max_iter=5000))
      ])
      model.fit(X_train,y_train)
      y_pred = model.predict(X_test)

      f1 = f1_score(y_test,y_pred)
      results[loss].append(f1)
```

```
plt.figure(figsize=(8,6))
for loss in losses:
    plt.plot(
        C_values,
        results[loss],
        label=f"Loss ={loss}"
    )

plt.xscale("log")
plt.xlabel("C (Regularization Strength)")
plt.ylabel("Test F1 Score")
plt.title("Impact of Loss and C value on SVM implementation")
plt.legend()
plt.grid(True)
plt.show()
```




```
from __future__ import division, print_function
import numpy as np
from sklearn import datasets, svm
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
%matplotlib inline
```

```
iris = datasets.load_iris()
X = iris.data[:,:2]
y = iris.target

X_train, X_test , y_train, y_test = train_test_split(X,y,test_size=0.25,random_state=42)
```

```
def evaluate_on_test_data(model=None):
  predictions = model.predict(X_test)
  correct_classifications = 0
  for i in range(len(predictions)):
    if predictions[i] == y_test[i]:
      correct_classifications += 1
  accuracy = 100*correct_classifications/len(y_test)
  return accuracy
```

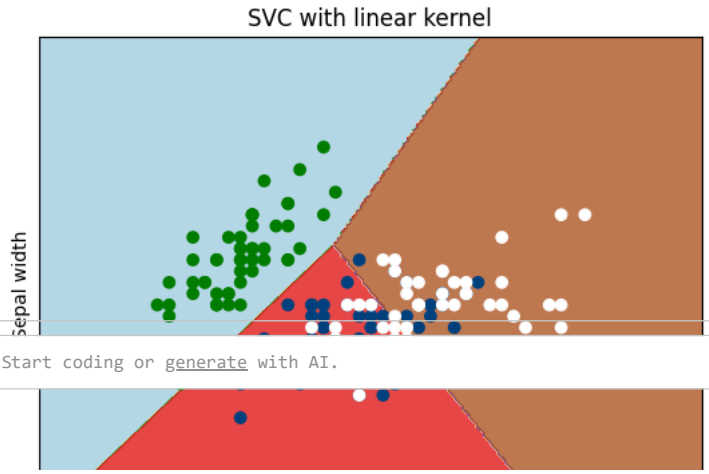
```
kernels = ('linear','poly','rbf')
accuracies = []
for index, kernel in enumerate(kernels):
  model = svm.SVC(kernel=kernel)
  model.fit(X_train, y_train)
  acc = evaluate_on_test_data(model)
  accuracies.append(acc)
  print("{} % accuracy obtained with kernel = {}".format(acc,kernel))
```

```
84.21052631578948 % accuracy obtained with kernel = linear
76.3157894736842 % accuracy obtained with kernel = poly
84.21052631578948 % accuracy obtained with kernel = rbf
```

```
svc = svm.SVC(kernel='linear').fit(X_train,y_train)
rbf_svc = svm.SVC(kernel='rbf',gamma=0.7).fit(X_train,y_train)
poly_svc = svm.SVC(kernel='poly',degree=3).fit(X_train,y_train)
```

```
#Create a mesh to plot in
h = .02  # step size in the mesh
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

#Define title for the plots
titles = ['SVC with linear kernel',
```

```
            'SVC with RBF kernel',
            'SVC with polynomial (degree 3) kernel']


for i, clf in enumerate((svc, rbf_svc, poly_svc)):
    # Plot the decision boundary. For that, we will assign a color to each
    # point in the mesh [x_min, m_max]x[y_min, y_max].
    plt.figure(i)

    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])

    # Put the result into a color plot
    Z = Z.reshape(xx.shape)
    plt.contourf(xx, yy, Z, cmap=plt.cm.Paired, alpha=0.8)

    # Plot also the training points
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.ocean)
    plt.xlabel('Sepal length')
    plt.ylabel('Sepal width')
    plt.xlim(xx.min(), xx.max())
    plt.ylim(yy.min(), yy.max())
    plt.xticks(())
    plt.yticks(())
    plt.title(titles[i])

plt.show()
```



Start coding or generate with AI.